



Baptiste Cojean & Gatien Auffret

ESIR 1 INFO

TP - Optimisation

13/05/2025

Sommaire et liste des figures :

I - Introduction.....	2
II - Calcul du gradient.....	3
II - Calcul de la matrice hessienne.....	3
III - Implémentation de l'algorithme du gradient.....	4
Figure 1 - Courbes d'erreur en fonction du nombre d'itérations.....	7
V - Conclusion :	7

I - Introduction

Lors de ce TP d'optimisation, nous allons résoudre un problème de régression parabolique en cherchant les coefficients de notre parabole afin de mieux approximer notre ensemble de points.

II - Calcul du gradient

On a :

$$J(p) = \frac{1}{2} \langle A p, p \rangle - \langle k, p \rangle$$

Dans le Matrix CookBook, on a la formule suivante :

$$\nabla_{\mathbf{x}} f = \frac{\partial f}{\partial \mathbf{x}} = (\mathbf{A} + \mathbf{A}^T) \mathbf{x} + \mathbf{b}$$

On calcul donc le gradient de J avec cette formule, on a donc :

$$J(p) = \frac{1}{2} p^T A^T p - k^T p = \frac{1}{2} p^T A p - k^T p$$

D'après la formule, on trouve :

$$\begin{aligned} \vec{\nabla} J(p) &= \frac{1}{2} (A + A^T) p - k \\ &= \frac{1}{2} (2 \times A) p - k \\ &= A p - k \end{aligned}$$

On en déduit donc que :

$$p^* = A^{-1} (\vec{\nabla} J(p) + K)$$

Au point minimum de J(p), $\vec{\nabla} J(p) = \vec{0}$

Donc : $p^* = A^{-1} K$

II - Calcul de la matrice hessienne

Dans le Matrix CookBook, on a la formule suivante :

$$\frac{\partial^2 f}{\partial \mathbf{x} \partial \mathbf{x}^T} = \mathbf{A} + \mathbf{A}^T$$

On calcul donc la matrice hessienne de J :

$$\begin{aligned}
 H_j(p) &= \frac{1}{2} (A + A^T) \\
 &= \frac{1}{2} (2A) \\
 &= A
 \end{aligned}$$

On cherche maintenant à calculer numériquement les valeurs propres de la matrice A suivante :

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 2 & 3 \end{pmatrix}$$

En matlab, grâce à la fonction eig, on obtient les 3 valeurs propres suivantes :

```
0.3080
0.6431
5.0489
```

On remarque que toutes les valeurs propres sont positives, A est donc définie positive et par conséquent, $H_j(p)$ l'est aussi.

$H_j(p)$ est donc strictement convexe et son minimum, si elle en a un, est donc global.

III - Implémentation de l'algorithme du gradient

On écrit une fonction matlab implémentant l'algorithme du gradient à pas fixe :

```
%% Méthode du gradient à pas fixe
function [p, erreur, n_iter] = Gradientfixe(A, k, alpha, tol, itermax)
    erreur = [];
    n = length(k);
    p{1} = rand(n, 1);

    for i = 1:itermax
        g = gradient(A, p{i}, k);
        p{i+1} = p{i} - alpha * g;
        err = norm(p{i+1} - p{i});
        erreur(end+1) = err;

        if err < tol
            break;
        end
    end

    n_iter = i;
end
```

On implémente également une fonction implémentant l'algorithme du gradient à pas optimal :

```

%% Méthode du gradient à pas optimal
function [p, erreur, n_iter] = Gradientoptimal(A, k, tol, itermax)
    erreur = [];
    n = length(k);
    p{1} = rand(n, 1);

    for i = 1:itermax
        g = gradient(A, p{i}, k);

        a2 = (g') * A * g / 2;
        a1 = (k' * g) - (p{i}' * A * g / 2) - (g' * A * p{i} / 2);
        alpha = -a1 / (2 * a2);

        p{i+1} = p{i} - alpha * g;
        err = norm(p{i+1} - p{i});
        erreur(end+1) = err;

        if err < tol
            break;
        end
    end

    n_iter = i;
end

```

On écrit ensuite la main en matlab nous permettant de gérer l'affichage ainsi que les différentes constantes :

```

function main
    % Déclaration des optimals
    A = [1,1,1;
         1,2,2;
         1,2,3];
    k = [1;
         1;
         1];
    alpha = 0.01;
    tol = 1e-6;
    itermax = 1000;

    [p_const, erreur_const, n_iter_const] = Gradientfixe(A, k, alpha, tol, itermax);
    [p_var, erreur_var, n_iter_var] = Gradientoptimal(A, k, tol, itermax);

    % 4 cas
    [p1, e1, i1] = Gradientfixe(A, k, 0.005, tol, itermax);
    [p2, e2, i2] = Gradientfixe(A, k, 0.05, tol, itermax);
    [p3, e3, i3] = Gradientfixe(A, k, 0.1, tol, itermax);
    [p4, e4, i4] = Gradientoptimal(A, k, tol, itermax);

    % Affichage
    figure('Position', [100, 100, 1000, 700]);

    subplot(2,2,1);
    plot(e1);
    title(sprintf('Pas fixe 0.005 : %d itérations', i1));
    xlabel('Itération'); ylabel('Erreur');

    subplot(2,2,2);
    plot(e2);
    title(sprintf('Pas fixe 0.05 : %d itérations', i2));
    xlabel('Itération'); ylabel('Erreur');

    subplot(2,2,3);
    plot(e3);
    title(sprintf('Pas fixe 0.1 : %d itérations', i3));
    xlabel('Itération'); ylabel('Erreur');

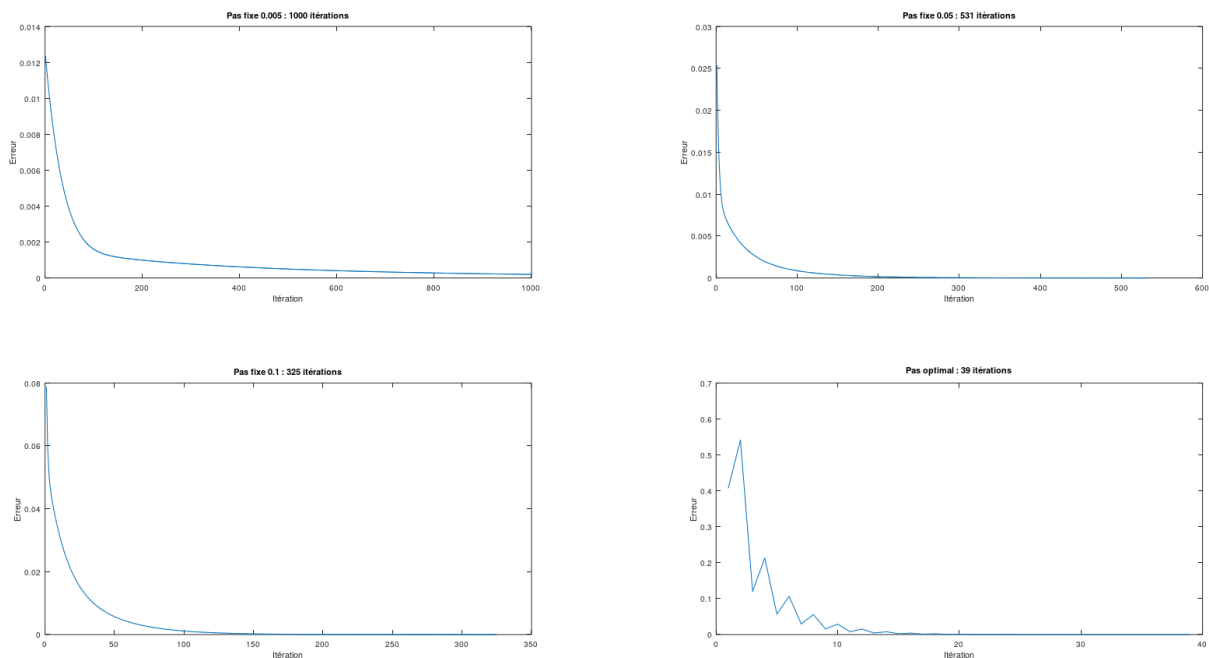
    subplot(2,2,4);
    plot(e4);
    title(sprintf('Pas optimal : %d itérations', i4));
    xlabel('Itération'); ylabel('Erreur');
end

```

On choisit à la place de 0.5 d'utiliser 0.1 car matlab ne supporte pas les valeurs trop grandes. En effet avec 0.5, $e3 = 7.5315e+181$ alors que matlab ne supporte rien au-dessus de $1e+38$.

On obtient le résultat suivant :

Figure 1 - Courbes d'erreur en fonction du nombre d'itérations



Pour un pas fixé à 0.005, on remarque que l'algorithme atteint 1000 itérations tout en convergeant, ce pas est donc trop petit pour atteindre notre tolérance.

Pour un pas fixé à 0.05, on remarque que l'algorithme atteint 597 itérations tout en convergeant. On atteint notre tolérance mais de manière toujours lente.

Pour un pas fixé à 0.1, on remarque que l'algorithme atteint 334 itérations tout en convergeant. C'est mieux que le reste mais ça reste toujours assez lent.

A l'inverse, pour un pas optimal, on remarque que l'algorithme atteint seulement 29 itération ce qui est très bon par rapport aux pas fixés.

V - Conclusion :

On en conclut donc que l'algorithme du gradient est bien plus efficace pour un pas optimal variable que pour un pas fixé.